Annals of Operations Research 42(1993)275–312

Feature extraction algorithms for constrained global optimization II. Batch process scheduling application

Athanasios G. Tsirukis and Gintaras V. Reklaitis
School of Chemical Engineering, Purdue University, West Lafayette, IN 47907, USA

Abstract

The feature extraction algorithms developed in part I of this series are applied to solve nonconvex mixed integer nonlinear programming problems which arise in the optimal scheduling of multipurpose chemical plants. A general formulation of the multipurpose plant scheduling problem is developed which considers the allocation of plant equipment and secondary, limited resources to recipe operations so as to satisfy given production requirements while minimizing cost. Results obtained with a test example involving 135 binary and 922 continuous variables show that the successive refinement strategy is effective in identifying dominant regions of the solution space. Furthermore it is shown that multiple moment based characterization methods are superior to the interval analysis method reported in the literature. Trials using a second, larger nonlinear test problem involving 356 binary and 2402 continuous variables demonstrate that the focused successive refinement strategy is more efficient than a constant resolution strategy which employs genetic algorithm constructions. Although the conventional genetic algorithm can be significantly improved by introducing a heuristic mutation strategy which increases the likelihood of constant feasibility, the successive refinement strategy remains dominant. These studies demonstrate that the feature extraction strategy employing successive refinements and relatively low order moment based region characterization methods, offers an effective approach to solving an important class of large scale MINLP problems with multiple local optima.

1. Introduction

In the first paper of this series we introduced the mathematical foundations of an approach to global optimization in constrained domains, called the feature extraction algorithms. The method is designed to screen-out inefficient decision subspaces in the domain of the optimization problem, guiding the decision-making towards more promising alternatives. The proposed solution methodology attempts to identify the topological characteristics of the objective and feasibility functions, by providing estimates of their average values inside any subset of the problem space. Using the implicit statistical framework of the optimization problem, we

J.C. Baltzer AG, Science Publishers

based the estimation of the objective and feasibility functions on the appropriate probabilistic computations. The statistical dependence of the problem random variables necessitates the relaxation of the problem semantics. The estimation procedure is shown to be asymptotically accurate, hence the information quality is increased by partitioning the problem domain into a large number of hyper-rectangles. To combat the combinatorial complexity of the domain partitioning problem, we introduce two alternative partitioning schemes. The first, called the successive refinement strategy, progressively increases the information quality by forming appropriate coverings of the feasible region. The second method, called the constant resolution strategy, employs the dynamics of a genetic algorithm in an effort to identify the most efficient problem subspaces.

In the present paper we will introduce an appropriate test methodology that examines the ability of feature extraction algorithms to identify dominant subspaces in reasonable computation time. The methodology is applied to a series of non-convex, combinatorially complex problems arising from multipurpose batch process scheduling applications. The proposed strategy will be compared to existing optimization algorithms.

2. Scheduling problem – definitions and formulation

The scheduling of a multipurpose batch chemical plant will constitute the test basis for the feature extraction approach. An introduction to the chemical plant scheduling problem and a review of the recent research activity in the area can be found in [7] and more extensively in [8]. The problem instance that will be examined in the present paper is concerned with the optimal allocation of plant equipment and secondary resources to a set of physical and chemical operations that are dictated by the product recipes and which are required to ensure satisfaction of given production quotas. The following definitions are of central importance for this scheduling problem:

Product: Denoted by p, $p = 1,2,\ldots,P$. These products are produced concurrently in the multipurpose batch chemical plant. The production of product p is denoted by q_p.

Equipment item: Denoted by e, $e = 1,2,\ldots,E$. V_e represents the volume of e, if it is a batch processing unit, or the processing rate, if it is a semicontinuous one.

Resource: Denoted by s, $s = 1,2,\ldots,S$. R_s denotes the maximum resource availability.

Task: A *recipe task*, (p,m), is the mth physical/chemical operation, required to complete the production of product p. The *recipe structure* RC_p contains the set of tasks necessary for the production of product p. It is represented by a graph structure whose nodes are recipe tasks and whose directed arcs define precedence relations among tasks. The *Production task*, (p,m), refers to the realization of a certain recipe task (p,m). Associated with each task there exists a set of equipment items, E_{pm},

able to process it, along with the corresponding batch size dependent nonlinear relation for the processing time, $f_{pme}(B)$, and a set of necessary secondary resources, R^{spme}, accompanied by the corresponding batch size dependent nonlinear relation for the resource consumption, $g_{spme}(B)$. The size factor of recipe task (p,m) produced in equipment e is S_{pme}. If an equipment of type e is assigned to the processing of a certain production task, (p,m), then a periodic production pattern is imposed and the quantity of product p processed in equipment e at a given time-instance is called the batch size, B_{pme}. The time necessary to complete the processing of this batch is denoted by t_{pme}, and is given by:

$$t_{pme} = f_{pme}(B_{pme}). \tag{1}$$

The corresponding consumption of resource s is denoted by r_{spme} and is given by:

$$r_{spme} = g_{spme}(B_{pme}). \tag{2}$$

Group: The group, G_{pm}, consists of a set of units assigned to the processing (again, under a periodic pattern) of a particular production task (p,m), $U_{pm} \subseteq E_{pm}$, the quantity of product p processed at a given time-instance, called the group batch size, B^G_{pm}, the time necessary for the production of a batch, called group cycle time, t^G_{pm} and the total consumption of resource s by all units in U_{pm}, r^G_{spm}. The equipment items in U_{pm} can be arranged in a parallel in-phase operating mode [7].

$$G_{pm} = (U_{pm}, B^G_{pm}, t^G_{pm}).$$

Production line: PL_p consists of the set of groups assigned to all production tasks of a given product, p, the quantity of product p at a given time-instance, called line batch size, B^L_p, the time necessary for the production of a batch, called line cycle time, t^L_p, and the total consumption of resource s by all groups G_{pm}, r^L_{sp}:

$$PL_p = \left(\left\{G_{pm}, m \in RC_p\right\}, B^L_p, t^L_p, \left\{r^L_{sp}, s = 1,2,\ldots,S\right\}\right).$$

Campaign structure: A campaign structure CS consists of the set of production lines assigned to all products p and the total consumption of resource s by all production lines r^c_s:

$$CS = \left(\left\{PL_p, p = 1,2,\ldots,P\right\}, \left\{r^L_{sp}, s = 1,2,\ldots,S\right\}\right).$$

The time that is necessary to complete all production requirements is called the campaign length or makespan.

Using the above notation, the equipment and resource assignment problem is stated as follows:

Given:

Equipment information: For each equipment item e, $e = 1,2,\ldots,E$, the volume or processing rate V_e.

Resource information: For each resource s, $s = 1,2,\ldots,S$, the maximum availability, R_s.

Production information: For each product p, the production requirement q_p.

Production recipe information: The production recipe information includes:

- RC_p, $p = 1,\ldots,P$;
- E_{pm}, $p = 1,\ldots,P$, $m \in RC_p$;
- f_{pme}, $p = 1,\ldots,P$, $m \in RC_p$, $e \in E_{pm}$;
- $g_{spme}(B)$, $s = 1,\ldots,S$, $p = 1,\ldots,P$, $m \in RC_p$, $e \in E_{pm}$;
- S_{pme}, $p = 1,\ldots,P$, $m \in RC_p$, $e \in E_{pm}$.

Cost: Makespan or any other measure of economic performance.

Determine: A campaign structure *CS* that minimizes *Cost*, so that all products are produced and all the physical, logical and production-dependent constraints are satisfied.

This equipment and resource assignment problem can be formulated as an optimization problem involving binary and continuous decision variables (MINLP). The constraint set consists of six principal subsets.

ASSIGNMENT AND CONNECTIVITY CONSTRAINTS

The structural decisions representing the assignment of equipment items to production tasks are handled by the following binary vector:

$$X_{pme} = \begin{cases} 1 & \text{if task } m \text{ of product } p \text{ is processed by unit } e, \\ 0 & \text{otherwise.} \end{cases} \tag{3}$$

Since all products must be satisfied, each product task should be assigned to at least one equipment item:

$$\sum_{e \in E_{pm}} X_{pme} \geq 1, \qquad m \in RC_p. \tag{4}$$

According to the following constraints, equipment item e can be assigned to at most one processing group:

$$\sum_{(p,m)\in P_e} X_{pme} \le 1, \qquad e = 1, \ldots, E, \tag{5}$$

where the set P_e records all the production tasks that can be processed in unit e:

$$P_e = \{(p,m) \mid e \in E_{pm}\}.$$

BATCH SIZE DEFINITION CONSTRAINTS

The quantity of task (p,m) produced in equipment e is related to the binary decision variable X_{pme}:

$$B_{pme} \le X_{pme} B^U_{pme}, \tag{6}$$

$$B_{pme} \ge X_{pme} B^L_{pme}, \tag{7}$$

where B^U_{pme} and B^L_{pme} are appropriate bounds that satisfy the following physical constraints:

$$B^U_{pme} \le \frac{V_e}{S_{pme}}, \tag{8}$$

$$B^L_{pme} \ge 0. \tag{9}$$

The group batch size is equal to the sum of batch sizes in the individual units:

$$B^G_{pm} = \sum_{e\in E_{pm}} B_{pme}. \tag{10}$$

The production line batch size is equal to the minimum group batch size among all production tasks of the same product:

$$B^L_p = \min_{m\in RC_p} \{B^G_{pm}\}. \tag{11}$$

PRODUCTION DEMAND CONSTRAINTS

Since all production requirements must be satisfied, the number of batches of product p is easily calculated from the following constraint:

$$n_p = \frac{q_p}{B^L_p}, \qquad p = 1, \ldots, P. \tag{12}$$

RESOURCE UTILIZATION CONSTRAINTS

The amount of secondary resource s utilized during the production of batch B_{pme} is determined by the functional form g_{pme} provided by the recipe information, which in the present problem is assumed to take a posynomial form:

$$r_{spme} = \delta_{spme} X_{pme} + \eta_{pme} B_{pme}^{\theta_{pme}}. \tag{13}$$

The group and production line resource utilizations are easily computed, using the following sets of constraints:

$$r_{spm}^{G} = \sum_{e \in E_{pme}} r_{spme}, \tag{14}$$

$$r_{sp}^{L} = \sum_{m \in RC_p} r_{spm}^{G}. \tag{15}$$

The total utilization of resource s is constrained by the resource availability level, R_s:

$$\sum_{p=1}^{P} r_{ps}^{L} \leq R_s. \tag{16}$$

CYCLE TIME DEFINITION CONSTRAINTS

The batch processing time is determined by the functional form f_{pme} provided by the recipe information, which in the present problem is assumed to take a posynomial form:

$$t_{pme} = \alpha_{pme} X_{pme} + \beta_{pme} B_{pme}^{\gamma_{pme}}. \tag{17}$$

The group cycle time is equal to the maximum batch processing time among all equipment items assigned to the same production task:

$$t_{pm}^{G} = \max_{e \in E_{pm}} \{t_{pme}\}. \tag{18}$$

The maximum group cycle time determines the production line cycle time:

$$t_{p}^{L} = \max_{m \in RC_p} \{t_{pm}^{G}\}. \tag{19}$$

COMPLETION TIME AND MAKESPAN CALCULATION

The completion time of product p is computed as follows:

$$T_p = n_p t_p^L, \quad p = 1,2,\ldots,P. \tag{20}$$

The makespan (objective function) is equal to the maximum completion time

$$Cost = \max_p \{T_p\}. \tag{21}$$

The mathematical formulation is completed by a set of direct and derived variable bounds. The form of the optimization model reveals the nonlinear and combinatorial nature of the corresponding search space. The nonlinear constraints (12), (13) and (17) are non-convex and can not be convexified by exponential transformation.

Since feature extraction algorithms will be used for the solution of the equipment and resource assignment problem, the problem formulation must be translated to a form processable by the solver. A careful study of the model structure leads to the identification of two different classes of constraints:

(i) The first set of constraints perform the objective function calculation ((6)–(12) and (17)–(21)). Figure 1 depicts the graph that describes the calculation of the objective function: if specific values are assigned to the independent variables B_{pme} and X_{pme}, then the values of the intermediate variables and of the objective function are easily computed. The objective graph is clearly not separable, due to the utilization of batch size information in the calculation of processing and cycle times. The selected relaxation scheme decomposes the tree-structure at the lower batch size level, by introducing statistically independent copies of the elementary batch size random variables B_{pme} and the binary random variables X^{pme}. The relaxed objective function form is shown in fig. 2. Notice that each iteration of the algorithm requires the estimation of probability distributions for every variable that appears in fig. 2.

(ii) The second class of constraints express the physical and logical relations of the problem ((4), (5) and (14)–(16)). The knowledge of the constraint probability distribution functions is not essential. Interval analysis methods are employed in the examination of the constraint feasibility, following the guidelines given in [9].

The set of independent decisions in the equipment and resource assignment problem contains the binary assignment variables X_{pme}, (3), and the batch-size bounds B_{pme}^L, B_{pme}^U, (8) and (9) respectively. Of those, the binary decision variables are considered more important and the domain partitioning effort will be focused on them, whereas the latter will not be refined further. Their values are given by eqs. (8) and (9). This decision was based on the following two observations:

(a) The instantiation of a binary decision variable may significantly influence the characteristics and the objective quality of the, as yet incomplete, schedule. If a decision is made to assign a certain equipment item to a particular production task, then this equipment item is unavailable for the other tasks that are processed in parallel (5). Therefore the instantiation of a binary

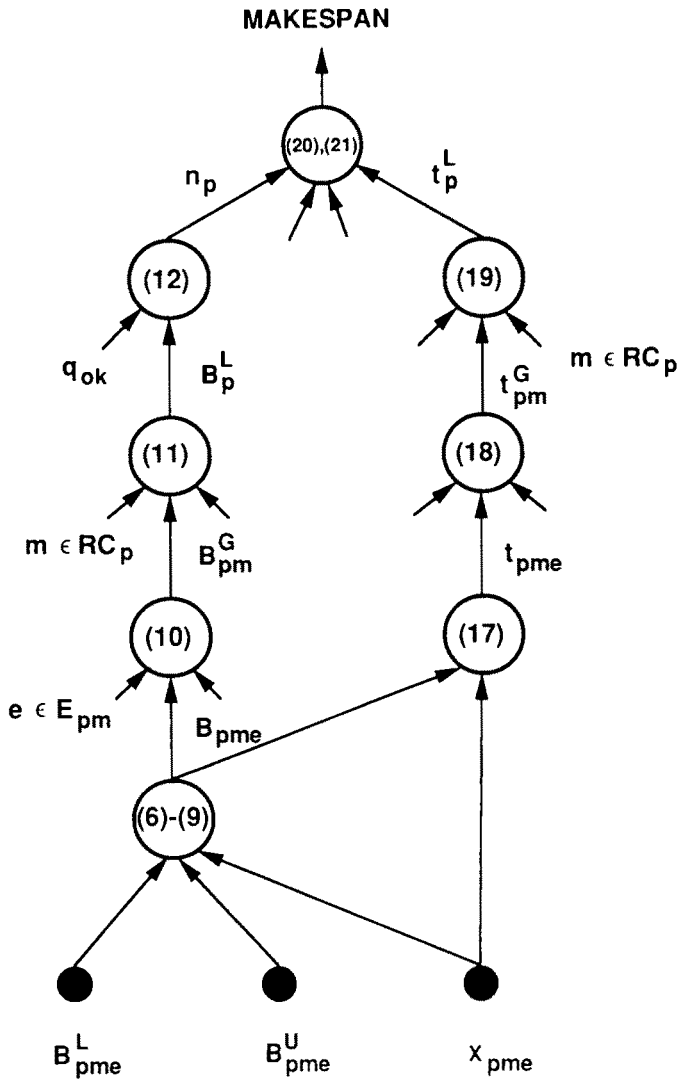

Fig. 1. Computation of makespan.

variable may influence a number of other uninstantiated decisions. This is not true for the upper and lower bounds on the batch size. A refinement of those values does not significantly influence the assignment decisions. On the contrary, eqs. (5) and (6) indicate the instantiation of a binary decision variable may turn some of the batch size bounds useless.

(b) In the successive refinement approach, each iteration results in a refinement of the feasible region cover set and a subsequent increase of the cardinality. For problems of practical size the number of distinct hyper-rectangles rises

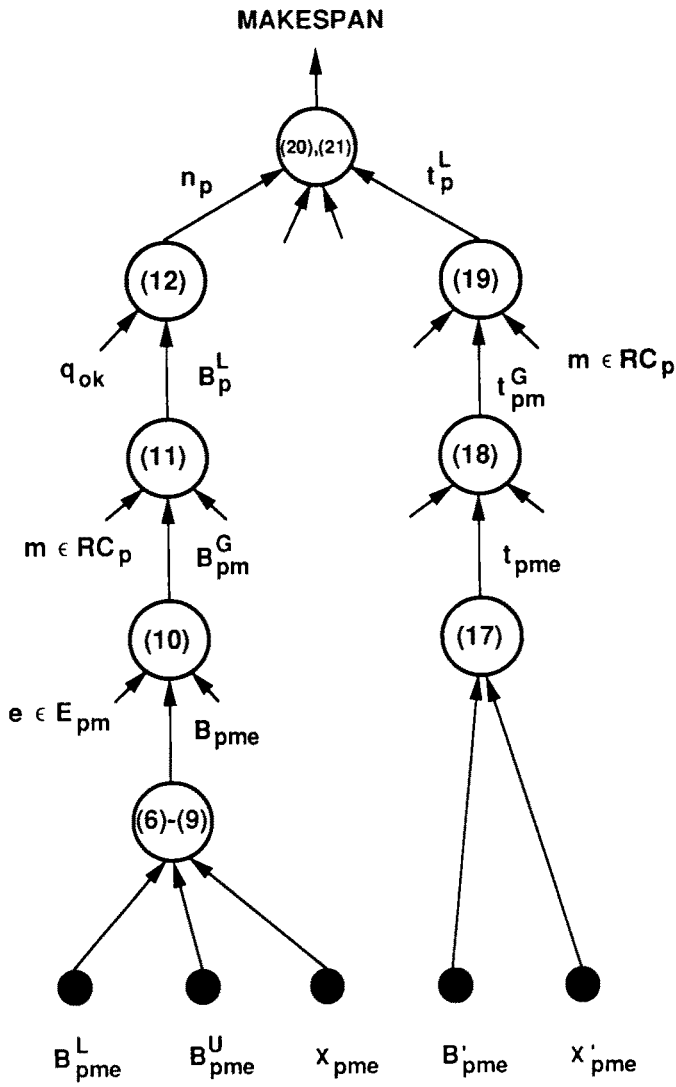

Fig. 2. Relaxed computation of makespan.

to thousands, and thus the working memory of a computers can quickly be consumed. If we decide to refine both the X_{pme} and the B^L_{pme}, B^U_{pme} independent variables, then each p, m, e combination consumes 1 bit for the binary variables and 64 bits for the bounds (in floating point representation), or total of 65 computer memory bits. Alternatively, if the batch size bounds are left uninstantiated, each p, m, e combination consumes a single bit for the binary decision variable. The latter decision significantly increases the efficiency of the utilized computing resources.

As was underlined in the previous section, each iteration of the successive refinement strategy results in an increase in the cardinality of the cover set. It is clear that the memory limitations of the computing system pose an upper bound on the number of hyper-rectangles retained in the partition set. This problem is accommodated by forcing the successive refinement strategy to simulate a focused-search algorithm: The cardinality of the covering set is kept below a prespecified limit. If, after a number of iterations, the limit is exceeded, an appropriate number of inefficient hyper-rectangles is removed. The resulting decrease of the cover volume per iteration guarantees the termination of the algorithm in a finite number of iterations. The number of iterations can be controlled by appropriately controlling the upper limit on the cardinality of the cover set.

The proposed solution methodology will be applied to the scheduling of an example multipurpose batch chemical plant, whose structure is adapted from Wellons and Reklaitis [10]. The plant consists of 9 equipment types as detailed in table 1.

Table 1

Description of plant equipment items.

Equipment type	Volume
Reactor U1	2500
Reactor U2	6300
Reactor V1	2500
Reactor V2	4000
Reactor D	4000
Filter S	300
Filter P	2000
Dryer T	1000
Dryer P	1000

Seven different products are produced in the plant, A through G. The corresponding product recipes contain on the average 6 processing tasks. Each processing task can be performed, on the average, by 9 different equipment items. The processing times are posynomial functions of the batch size, of the form (1). Size factors, processing time parameters and production recipe information for products A through F are given in tables 2 through 8. In the description of the multipurpose batch plant we have included four renewable and continuously divisible secondary resources, S 1, S 2, S 3 and S 4. The resource utilization rates are, again, posynomial functions of the batch size, described by (2). The corresponding δ, η and θ constants are presented in table 9. The resource utilization will be allowed to vary depending on the examined problem instance. Finally, the approximation of the dependent probability distribution functions was based on the first four moments, unless otherwise specified.

Table 2

Recipe information for product A.

Task	Size factor S	Processing time constants α	β	γ	Feasible equip. types
1	2.74	15.00	1.72e – 2	0.865	RD
2	0.26	2.00	6.12e – 6	2.000	FS
3	5.85	16.00	3.64e – 2	0.823	All R
4	1.78	0.00	4.28e – 3	1.000	All R
5	1.98	0.00	4.12e – 2	0.808	RV 1, RV 2
6	1.58	1.00	4.28e – 5	2.000	FP
7	1.444	2.00	3.70e – 2	1.000	DT

Table 3

Recipe information for product B.

Task	Size factor S	Processing time constants α	β	γ	Feasible equip. types
1	4.48	2.00	4.00e – 2	0.865	RD
2	0.55	1.00	6.12e – 6	2.000	FS
3	5.51	0.00	3.64e – 2	0.823	RV 1, RV 2
4	2.63	1.00	4.28e – 3	1.000	All R
5	8.62	3.00	4.12e – 2	0.808	All R
6	1.83	2.00	4.28e – 5	2.000	FP
7	1.66	4.00	3.70e – 2	1.000	DT

Table 4

Recipe information for product C.

Task	Size factor S	Processing time constants α	β	γ	Feasible equip. types
1	4.70	2.00	4.00e – 2	0.737	RD
2	0.31	1.00	4.08e – 6	2.000	FS
3	4.61	0.00	2.27e – 2	0.823	RV 1, RV 2
4	3.48	0.00	1.36e – 2	0.823	All R
5	7.55	2.50	3.26e – 2	0.830	All R
6	1.83	2.00	1.83e – 5	2.000	FP
7	1.66	4.00	1.71e – 2	1.000	DT

Table 5

Recipe information for product D.

Task	Size factor S	Processing time constants α	β	γ	Feasible equip. types
1	1.18	0.00	1.89e – 2	0.768	All R
2	7.17	2.00	1.55e – 2	0.927	All R
3	8.59	2.50	4.26e – 2	0.784	All R
4	1.54	2.00	1.13e – 5	2.000	FP
5	1.40	4.00	1.50e – 2	1.000	DP

Table 6

Recipe information for product E.

Task	Size factor S	Processing time constants α	β	γ	Feasible equip. types
1	7.98	3.50	8.00e – 4	1.487	All R
2	3.15	1.00	4.08e – 5	2.000	FP
3	2.86	2.00	3.70e – 2	1.000	DP

Table 7

Recipe information for product F.

Task	Size factor S	Processing time constants α	β	γ	Feasible equip. types
1	7.54	9.00	2.00e – 2	0.830	All R
2	8.01	0.00	1.22e – 1	0.704	RV1, RV2
3	2.95	1.00	1.87e – 5	2.000	FP
4	2.65	2.00	2.50e – 2	1.000	DP

Table 8

Recipe information for product G.

Task	Size factor S	Processing time constants α	β	γ	Feasible equip. types
1	5.32	4.00	5.92e – 2	0.756	RD
2	0.15	1.00	1.97e – 2	2.000	FS
3	14.65	4.00	5.88e – 2	0.804	All R
4	1.11	0.00	3.66e – 2	0.768	All R
5	3.17	1.00	3.45e – 5	2.000	FP
6	2.88	2.00	1.77e – 2	1.000	DT

Table 9

Resource utilization parameters.

Equipment type	Resource	δ	η	θ
Reactor	S1	3.00	1.36e – 2	0.830
	S2	3.00	2.00e – 3	1.200
	S3	2.00	1.00e – 3	1.100
	S4	3.00	2.00e – 2	0.250
Filter	S3	2.00	1.83e – 5	2.000
	S4	2.00	3.20e – 2	0.500
Dryer	S1	3.00	3.00e – 2	1.000
	S2	1.00	3.00e – 2	0.750
	S3	2.00	5.00e – 4	1.000
	S4	5.00	1.55e – 1	0.300

3. Scheduling test problem 1

The purpose of the first test problem is to investigate the ability of feature extraction algorithms to discover globally optimal solutions. It is well known that the global optimization problem is NP-hard [4]. Therefore, any increase in the problem dimensionality is accompanied by an exponential increase in the amount of computational resources (cpu power and/or time) needed to identify the global optimum. Many global optimization algorithms that perform well on small problem instances fail to converge in reasonable computation times for larger test cases. Since the proposed scheduling algorithm was designed to accommodate problems

of practical size and to provide a good solution in reasonable computational times, test problems of non-trivial dimensionality were created.

The first test problem involves the assignment of equipment and resources to three products that will be processed concurrently. Table 10 describes the production requirements. The plant contains 32 processing units, detailed in table 11. The resource availability levels are given in table 12. The objective of the scheduling efforts is to minimize the makespan. This problem can be formulated as a MINLP involving 135 binary decision variables, 922 continuous variables and 1427 linear and nonlinear constraints.

Table 10

Production requirements for test problem 1.

Product	Quantity
C	32000
F	47000
G	20000

Table 11

Processing units for test problem 1.

Equipment type	Number of units
Reactor U1	3
Reactor U2	3
Reactor V1	3
Reactor V2	3
Reactor D	5
Filter S	2
Filter P	4
Dryer T	4
Dryer P	4

Table 12

Resource availability for test problem 1.

Resource	Availability
S1	1000
S2	1000
S3	1000
S4	1000

3.1. FEATURE EXTRACTION AND CLASSIFICATION

In order to be efficient, feature extraction algorithms are required to quickly identify suboptimal combinations by correctly classifying subspaces in the problem domain according to the quality of the contained local optima. The present section is devoted to the study of the classification properties of the feature algorithms.

An efficient classifier must possess two important properties: non-degeneracy and accuracy. A classifier is called degenerate if it assigns the same characterization measure to volumes with different global optimization characteristics. A degenerate algorithm is likely to remove an efficient subspace due to lack of information.

The study of the degeneracy properties of a feature extraction algorithm is based on the following methodology, which is called the degeneracy test: The domain of the test problem is partitioned by the successive refinement strategy into a large number of subspaces, close to the cardinality upper bound imposed by the memory limitations of the computing system. The algorithm characterizes the partition members and distributes them into a prespecified number of classes, based on the computed characterization measure. The results are presented in distribution diagrams depicting the number of hyper-rectangles in each class. If the classification is efficient, then a large number of classes is created, each containing a limited number of subspaces. Hence, the dominant class contains few alternatives which are easily examined in order to identify efficient solutions. If the classification is degenerate, then the decision subspaces are distributed into a limited number of classes. The dominant class contains a large number of alternatives whose examination requires increased computational effort. In addition, the possibility of removing efficient subspaces becomes significant.

The proposed methodology was applied to the first test problem. The domain was partitioned into 1000 subspaces which were subsequently characterized by the feature extraction algorithm, that was described in the previous section, and distributed into 50 classes. The corresponding distribution diagram is shown in fig. 3. Notice that the classes are ordered in decreasing objective quality. Obviously, the algorithm is non-degenerate. It creates a very smooth classification of the partition members. The dominant classes (e.g. the first one or two) are clearly separated from the core, containing a small number of alternatives. These characteristics significantly facilitate the decision-making process.

In addition to non-degeneracy an efficient classifier must be accurate, that is the characterization measure must correctly reflect the quality of a decision subspace and, most importantly, preserve the quality relations among different subspaces. In other words, if subspace A is superior to B, then one should be able to infer the same relation by comparing their corresponding measures. The asymptotic convergence properties of feature extraction algorithms guarantee accuracy in the limit of zero partition fineness. For practical applications requiring the use of a focused successive refinement strategy (section 2) accuracy is of a crucial importance even when the partition fineness is still large. Inaccurate classification can result in removing

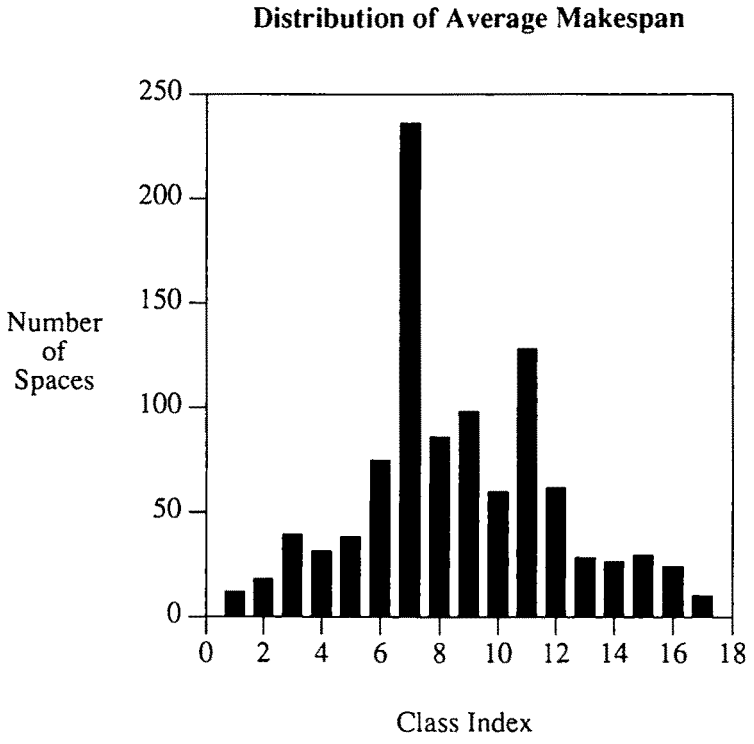

Fig. 3. Distribution Diagram for test problem 1.

efficient subspaces. Hence, an accurate procedure that will measure the classification properties of an algorithm must be devised.

If complete information about the optimization problem was available, one would easily compare the classification results computed by the algorithm (computed measure) for a number of test hyper-rectangles, against their real objective value average (absolute measure). Due to the complexity of the underlying statistical computation, the calculation of the exact population statistics is computationally intractable. Based on the hypothesis that subspaces with better objective average contain, on the average, better feasible solutions, we devised the following comparison scheme, called the accuracy test: A fine partition of the problem domain is formed and a small population of partition members is randomly selected and characterized by the feature extraction algorithm. Using the successive refinement strategy, each hyper-rectangle is searched until a locally optimal solution is identified in it. The objective function of the local optimum is chosen to serve as the objective quality measure of the corresponding subspace.

The results of the classification method are presented in the form of a classification diagram. Each hyper-rectangle is represented by a data point in the diagram, with the x-coordinate corresponding to the average makespan (characterization measure)

and the y-coordinate representing the locally optimal makespan (objective quality measure), both appropriately normalized for convenience. The classification ability of a feature extraction algorithm can lie anywhere between two extreme cases. In the first case, the classification ability is perfect. The characterization measure is proportional – with a positive constant of proportionality – to the objective function value of the global optimum in each hyper-rectangle. Hence, the data points lie on the diagonal of the diagram. In the second case the algorithm misclassifies the hyper-rectangles. Dominant subspaces are classified as inferior. The relation is, again, linear, with a negative proportionality constant.

An accurate measure of the classification ability is provided by the correlation coefficient of the data [5], which indicates the existence of a linear dependence in the data set. If the correlation coefficient is positive and close to 1, the classification is accurate and if it is negative and close to -1 the classification is inaccurate. If the correlation coefficient is close to 0, the algorithm is unable to extract any useful information, thus the classification is degenerate.

According to the classification methodology, the problem domain is partitioned into 1000 pieces and a population of 17 randomly hyper-rectangles is formed. Each subspace is further refined until the discovery of a solution with completely instantiated binary decision variables. The reduced nonlinear optimization problem is solved by an appropriate mathematical programming algorithm. We employed the nonlinear optimizer MINOS 5.3 [3], executed under the mathematical representation language GAMS 2.25 (Generalized Algebraic Modeling Systems [2]). At this point, a feasible equipment and resource assignment has been identified. The corresponding classification diagram is shown in fig. 4. The correlation coefficient of 0.79 indicates that the computed performance measure is correlated with the objective measure.

The scheduler is able to classify the examined hyper-rectangles into three classes: the dominant ones, with average makespan ranging over [0, 0.5], the intermediate quality values that range over [0.5, 0.8] and the inefficient ones ranging over [0.8, 1]. The three scheduling classes are separated by dotted lines in fig. 4. This classification reflects a corresponding difference in scheduling philosophy. A careful examination of the problem data reveals the existence of four rate-limiting tasks: the first and second tasks of product F and the third task of product G. Notice that all four tasks compete for the same resources and equipment items. The parallel in-phase production mode significantly influences the production cycle times of the rate limiting tasks, which are characterized by large size factors. Therefore, the distribution of resources and equipment among the rate-limiting steps determines the quality of the resulting schedule. The class of inefficient hyperspaces is characterized by a premature assignment of a significant amount of resources and equipment to product G. The resulting limitation in production resources constrains the rates of the production tasks of F, leading to an increase in the makespan. In the second class of hyper-rectangles the level of production resources prematurely assigned to product G is smaller. Therefore, more production flexibility is allowed for product F and the makespan decreases on the average by 30 percent. Finally, the class of

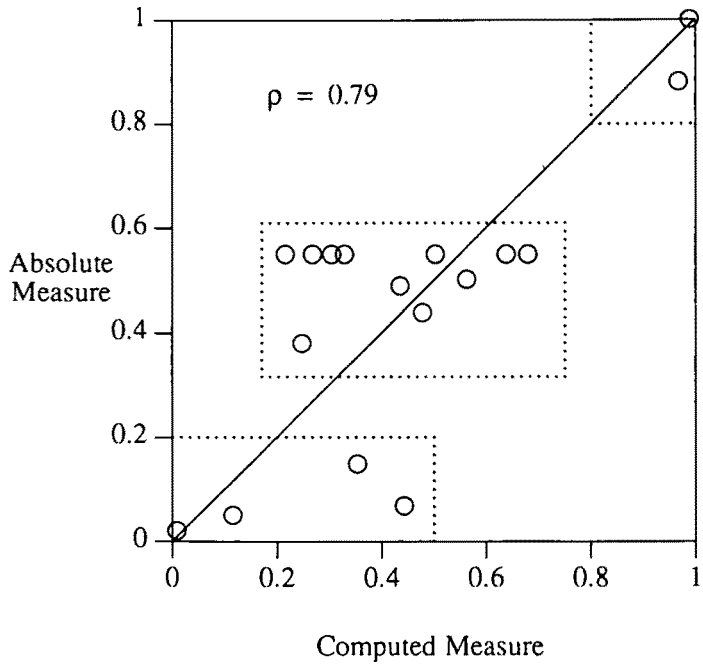

Fig. 4. Classification diagram – large volumes.

efficient hyper-rectangles is characterized by an initial balanced assignment of utilities and equipments to products F and G, accompanied by an equal balance in the corresponding production rates. At this point we have to stress the dependence of the above analysis on the problem structure (e.g. objective function, constraints) and the given data.

From the data in fig. 4, we notice that the average makespan can not induce a sharp classification of the hyper-rectangles whose average makespan ranges over the interval [0.2, 0.45]. This behaviour can be attributed to the following reasons:

(a) The classification relies on the feature extraction algorithm which approximates the volume properties. As a reminder, the approximation relaxes the problem semantics and utilizes only four moments in the computation of the dependent distributions.

(b) Each examined hyper-rectangle has a large volume, containing many distinct locally optimal solutions that exist because of the problem's combinatorial nature. It is possible that the averaging of the objective values over all possible decision states may obscure the existence of exceptional solutions. For example, hyper-rectangle A may contain schedules that are, on the average, superior to those of hyper-rectangle B, and still the best point of A may be inferior to that of B. The averaging error is expected to be mitigated by an increase in the partition fineness.

In order to study the effect on the scheduler's classification capability, we create a classification diagram for the reduced feasible spaces that were identified by the application of the successive refinement algorithm on each one of the original hyper-rectangle.

The reduced spaces are expected to contain only a few local optima. From the corresponding diagram in fig. 5 we anticipate the significant increase in the classification ability of the average maskespan. The three different scheduling classes are sharply separated.

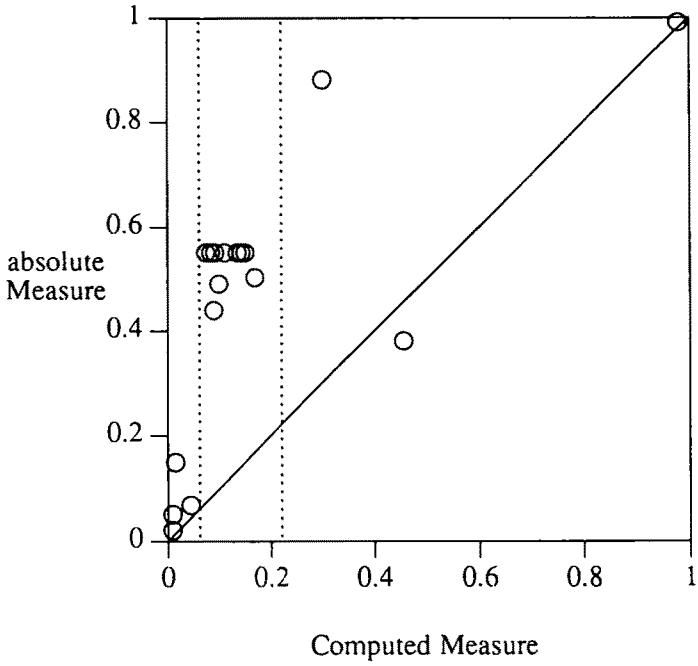

Fig. 5. Classification diagram – reduced volumes.

In order to provide an overview of the set of locally optimal solutions contained in the problem domain and to further underline the system's classification ability, we introduce in the population of fig. 5 a reference schedule that was generated by a random assignment of equipment to production tasks. Figure 6 depicts the augmented classification diagram, after being renormalized. The correlation coefficient of the augmented data set is 0.9, confirming the existence of correlation between the computed and the objective measure. The dashed rectangle in fig. 6 corresponds to the classification diagram in fig. 5. Under this more general perspective, the classification quality of the average makespan is even more evident. Figures 4 through 6 demonstrate the classification ability of the feature extraction algorithms. The most important classification property, namely the monotonic dependence of the objective average

on the local optimum quality is preserved and accurately monitored. Hence, the average makespan of a hyper-rectangle is highly correlated to its global optimization characteristics, even for gross partitions of the problem space.

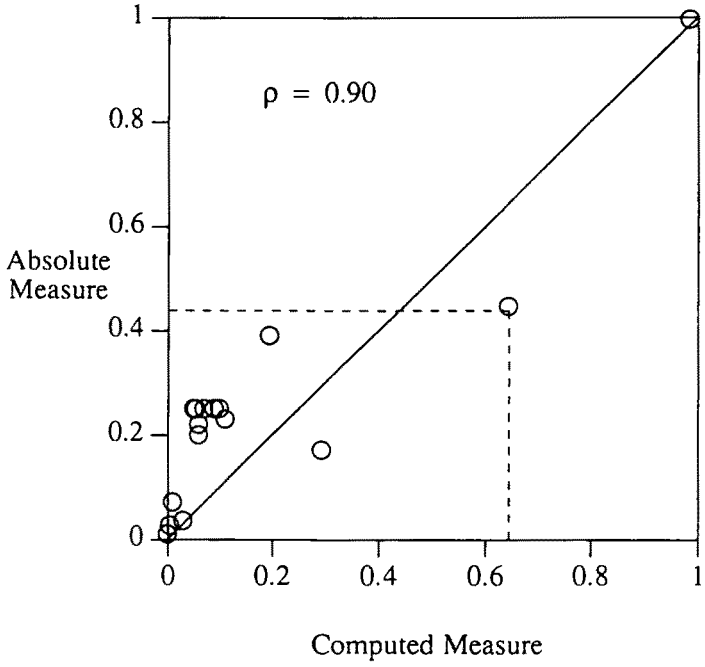

Fig. 6. Augmented population – Reduced volumes.

As was already mentioned, the successive refinement strategy searches each hyper-rectangle until a feasible solution is encountered. The refinement is applied to the hyper-rectangle with the best average makespan. Since the feasibility of the constraints is explicitly determined, the method is guaranteed to converge to a feasible schedule, if one exists. The results given in fig. 6 demonstrate the search efficiency. Even though each candidate hyper-rectangle contains many inefficient local schedules, such as the reference schedule that is located on the extreme point (1, 1) of the classification diagram in fig. 6, the strategy seems to anticipate the structural deficiencies of the search hyper-rectangle (e.g. initial overcommitment of production resources to nonlimiting tasks) and tries to compensate for them. As a result, the search converges to a set of reduced spaces that contain very efficient schedules. The best schedule that was generated during the detailed examination of the 17 hyper-rectangles corresponds to point (0, 0) in fig. 6. Its makespan is equal to 474 time units. The corresponding equipment assignment is shown in table 13. By contrast, the makespan of the reference schedule is equal to 1626 time units. The locally optimal solutions that were discovered by the successive refinement strategy in the 17 examined hyper-rectangles are given in table 14.

Table 13

Efficient equipment assignment.

Product	Task	Equipment items
C	1	RD
	2	FS
	3	RV2
	4	RU1
	5	RU2
	6	FP
	7	DT, DT
F	1	RU2, RD, RD
	2	RV1, RV1, RV2, RV2
	3	FP, FP
	4	DP, DP, DP, DP
G	1	RD
	2	FS
	3	RU2, RD
	4	RU1, RU1
	5	FP
	6	DT, DT

Table 14

Locally optimal solutions.

Subspace	Makespan
1	474
2	493
3	748
4	660
5	748
6	748
7	748
8	544
9	716
10	503
11	690
12	748
13	722
14	748
15	748
16	912
17	978
reference	1626

3.2. NUMBER OF MOMENTS AND INFORMATION QUALITY

In this section we will study the influence of the number of moments used in the approximation of the dependent probability distribution functions (PDF) on the classification ability of the feature extraction algorithms. The natural question arising from the study of section 4 in [9] is whether the information quality provided by the feature extraction algorithms is worth the extra computation expense compared to the information provided by the interval analysis method presented by Ratcheck [6]. Intuitively, one would expect that the descriptive power of a higher-moment approximation would be superior. The following example will shed some insight on this question.

EXAMPLE 1

Suppose that in a multipurpose batch plant there exists a product whose production recipe contains two processing tasks A and B. It is known that the processing time of task B is equal to 0.2 time units. Task A is processed in units of equipment type e, in a parallel in-phase mode of operation. The total batch size of A, B_T, ranges in the interval [0, 1]. Hence, if N units are available, then the unit batch size is given by:

$$B_e = \frac{B_T}{N}, \tag{24}$$

where N ranges in the set $\{1, 2, 3, 4, 5\}$. It is also known that the processing time of task A depends linearly on the batch size, as follows:

$$T_A = \frac{B_e}{2}. \tag{25}$$

The problem is to determine the cycle-time limiting task. Since the variables are uninstantiated, such an indication will be given by the probability that task A is limiting, or $Pr[T_A \geq 2]$. An estimate of this probability will be computed using interval analysis and feature extraction algorithms. The esimates will be compared against the analytical probability calculation.

Using the interval analysis methodology and eq. (24) the unit batch size is guaranteed to range in the interval:

$$B_e \in = \left[\frac{0}{5}, \frac{1}{1}\right] = [0,1].$$

The processing time will range in the interval:

$$T_A \in \frac{1}{2}[0,1] = [0,0.5].$$

The average processing time of task A is equal to 0.25. The probability that task A is the cycle-time limiting step is:

$$p = \frac{(0.5 - 0.2)}{(0.5 - 0.0)} = 0.6.$$

Hence, with 60 percent probability, A is the cycle-time limiting production task.

If we employ the feature extraction approach with first-moment approximations, then N is a discrete variable uniformly distributed over its corresponding range. The average unit batch size is detemined by the application of the theorem of total expectation, as described in [9]:

$$E[B_e] = \sum_{n=1}^{5} Pr[N = n]\, E\left[\frac{B_T}{n}\right]$$

$$= \frac{1}{5}\left(0.5 + \frac{0.5}{2} + \frac{0.5}{3} + \frac{0.5}{4} + \frac{0.5}{5}\right) = 0.22833.$$

Hence, the average processing time is $E[T_A] = 0.142$. The significant departure from the result obtained via the interval analysis method is already apparent. Using the maximum entropy method, the probability distribution function of T_A is approximated by an exponential cumulative distribution function, of the form:

$$Pr[T_A \leq t] = 1 - e^{-\lambda t}, \quad \lambda = \frac{1}{E[T_A]} = 8.791 \quad \Rightarrow$$

$$Pr[T_A \geq t] = 0.1734.$$

The information provided by the feature extraction algorithms leads us to the conclusion that task B is cycle-time limiting with 83 percent probability.

The analytical probability computation is based on the total probability theorem [5]:

$$Pr[T_A \geq 0.2] = \sum_{n=1}^{5} Pr[N = n] Pr[T_{A,N} \geq 0.2]$$

$$= \sum_{n=1}^{5} Pr[N = n] Pr\left[\frac{B_T}{2n} \geq 0.2\right]$$

$$= \frac{1}{5}(0.6 + 0.2 + 0 + 0 + 0) = 0.16.$$

Task *B* is cycle-time limiting with 84 percent probability. The superiority of the feature extraction algorithms is evident. The interval analysis does not provide enough information about the problem domain, thus leading to misclassification.

The remaining part of this section will be devoted to the comparison of the results obtained from the feature extraction approach and the interval analysis method. The first algorithm, whose results have already been analyzed, will be referred to as the 4-moment method. The interval analysis algorithm is described by Ratcheck [6]. For a given hyper-rectangle, the method provides upper and lower bounds on the objective function. Since no more information is available, the hyper-rectangle will be characterized by the average of the upper and lower bounds. This algorithm will be referred to as the 0-moment method, since it is equivalent to a 0-moment feature extraction algorithm. Both implementations employ identical bounding techniques and partitioning strategies.

We begin by a detailed comparison of the degeneracy properties of the two algorithms. Figure 7 contains the distribution diagram corresponding to the

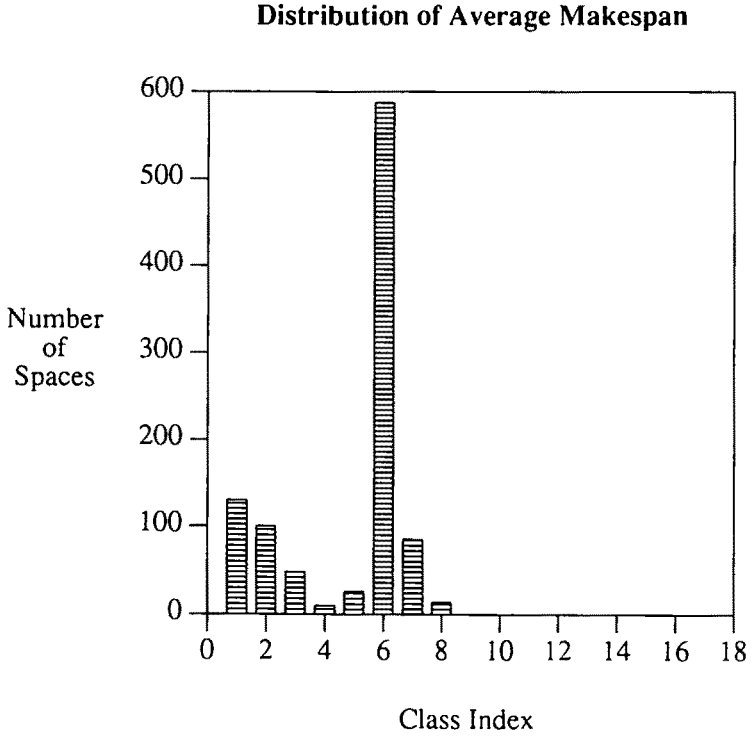

Fig. 7. Distribution diagram – 0 moments.

0-moment method, obtained by the procedure that was outlined in section 3.1. For ease of reference, the distribution diagram of the 4-moment method has been rescaled

and presented in fig. 8. Compared to the 4-moment method, interval analysis is more degenerate, distributing the partition members into a small number of classes. Notice that 60 percent of the partition members are accumulated in a single class.

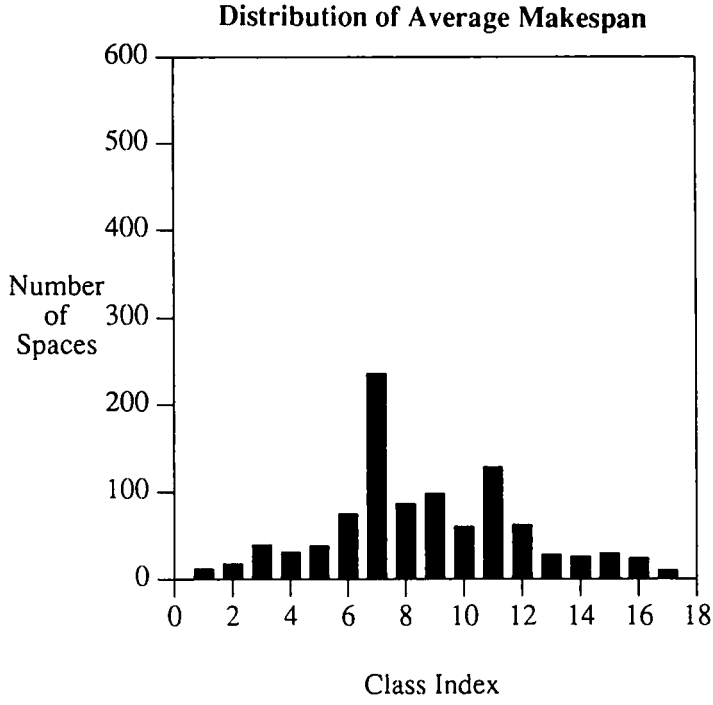

Fig. 8. Distribution diagram – 4 moments.

The degeneracy is even more obvious if we increase the prescribed bound on the number of classes from 50 to 200. This amounts to an increase in the resolution of the distribution diagram. Figure 9 contains the distribution diagrams of the most dominant 100 subspaces, as classified by the 4-moment and 0-moment algorithms. At this stage it is clear that interval analysis is degenerate and unable to provide any useful information about the relative quality of the first 100 members. By contrast, the 4-moment method creates a clear classification of the dominant population.

Since it is possible that the degeneracy in the results of the 0-moment method reflects possible symmetries in the geometry of the problem domain, while the classification created by the 4-moment method is inaccurate, the accuracy of both methods must be appropriately tested. From the population of 100 subspaces belonging to the first class of fig. 9 we randomly select 5 members, to which we apply the accuracy test, as outlined in section 3.1. The resulting classification diagrams are depicted in fig. 10. It is clear that the examined subspaces have different objective characteristics, containing solutions of significant diversity. Hence, the degeneracy

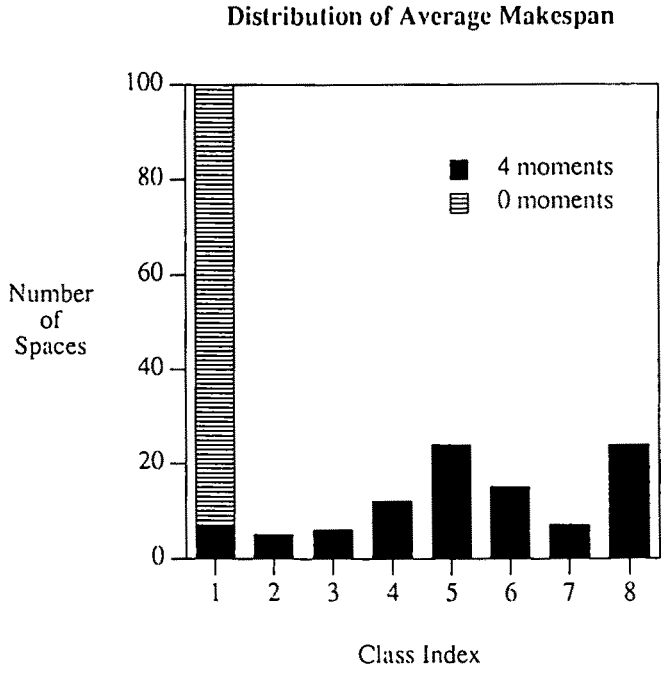

Fig. 9. Comparison of classification degeneracy.

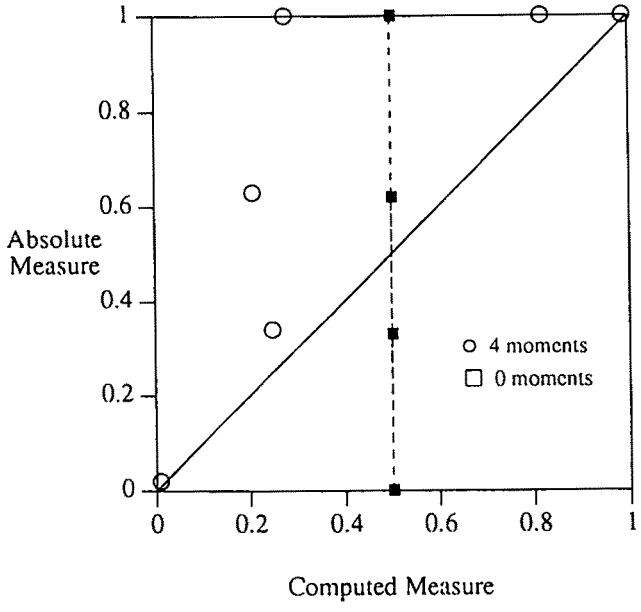

Fig. 10. Comparison of classification accuracy.

suggested by the 0-moment method is not real, rather it must be attributed to its inability to deduce significant information about the domain geometry at this level of partition fineness. The 4-moment method exhibits significantly superior classification ability, even though some indecision is apparent in the neighborhood of 2. Nevertheless, the algorithm is able to correctly identify the most and least efficient decision subspaces.

In the presence of a focused successive refinement strategy, in which inefficient hyper-rectangles are removed, the degeneracy of the classification criterion can lead to suboptimal solutions. In the last experiment of the present section we study the influence of degeneracy on the solution quality. We perform a number of tests involving the application of 4-moment and 0-moment algorithms to the first test problem. The domain is partitioned by the focused successive refinement strategy and different values for the cover set cardinality are examined. The search terminates when the volume of the cover space has been reduced to zero. At this point the cover set contains only fully instantiated solutions. The results are shown schematically in fig. 11. It is evident that the classification superiority of the 4-moment method leads to more efficient solutions, while interval analysis gets trapped in suboptimal decision spaces. Both methods are characterized by an initial phase of instability,

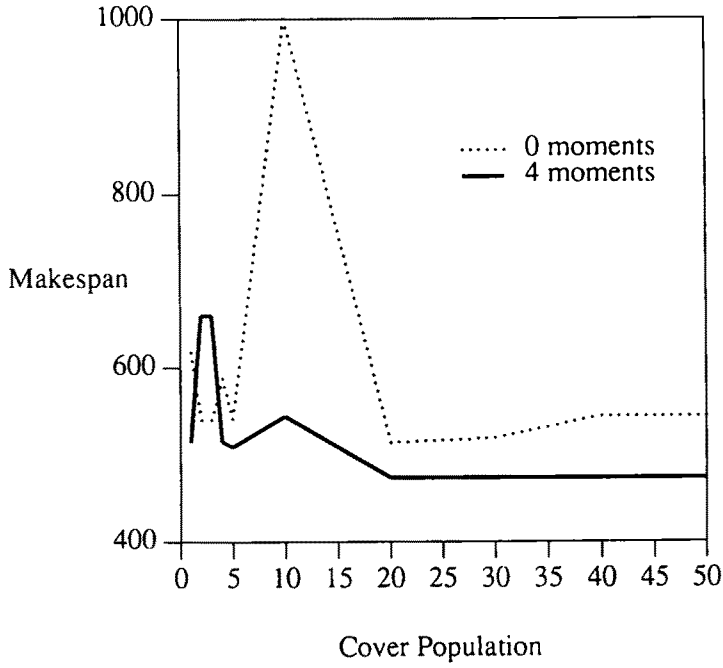

Fig. 11. Comparison of solution quality.

after which an increase in the cover set cardinality does not seem to influence the solution quality. The solutions obtained are summarized in tables 15 and 16, along

with the corresponding number of iterations. The computational requirements of both algorithms are reasonable. One thousand iterations of the 4-moment algorithm require 720 cpu seconds on a SUN SPARCstation 1, while the same number of iterations of the 0-moment method is completed in 110 cpu seconds.

Table 15

Scheduling results – 4-moments.

Cover set cardinality	Iterations	Makespan
1	55	515
2	93	660
3	128	660
4	187	515
5	221	509
10	450	544
20	820	474
30	1301	474
40	1620	474
50	1923	474

Table 16

Scheduling results – 0-moments.

Cover set cardinality	Iterations	Makespan
1	23	617
2	112	540
3	134	540
4	187	587
5	242	541
10	463	1031
20	948	514
30	1356	519
40	1904	544
50	2435	544

The makespan of the most efficient equipment assignment identified by the 4-moment method is equal to 474 time units. This is equivalent to the one contained in table 13. The best solution identified by the 0-moment method is presented in table 17. It is encouraging to notice that even the 0-moment method, whose classification abilities are moderate, is able to identify efficient solutions in a problem that has

Table 17

Equipment assignment – 0-moments.

Product	Task	Equipment items
C	1	RD
	2	FS
	3	RV2
	4	RU2, RD
	5	RU2
	6	FP
	7	DT, DT
F	1	RV1, RD, RD
	2	RV1, RV2, RV2
	3	FP, FP
	4	DP, DP, DP, DP
G	1	RD
	2	FS
	3	RU1, RU2
	4	RV1
	5	FP
	6	DT

non-trivial decisional complexity. The objective function values of the obtained solutions are within 10 percent of the most efficient solution obtained in all the scheduling experiments.

3.3. DISCUSSION

The analysis of the first test problem underlines the superior classification abilities of the multi-moment feature extraction algorithms. Therefore, high-quality solutions are discovered in very reasonable computation times. As expected, the increase in the quality of extracted information is accompanied by an increase in the computation time. The interval analysis method is almost seven times faster than the 4-moment feature extraction algorithm. It is clear that there exists a trade-off between the obtained solution quality and the invested computation time. The problem of determining an "optimal" number of moments is non-trivial. Nevertheless, the following considerations can be offered as guidelines during the design phase of a feature extraction algorithm:

(a) If the problem at hand is of significant combinatorial complexity and dimensionality, then it is probable that the focused successive refinement strategy will be quite efficient. The upper bound on the cover set cardinality

that is imposed by computer memory limitations necessitates the removal of inefficient subspaces. During the first iterations it is imperative that the feature extraction algorithm exhibits superior classification abilities, so as to minimize the possibility of removing an efficient problem subspace. This is equivalent to selecting a multi-moment feature extraction algorithm. When the partition fineness is significantly reduced, then any feature extraction algorithm is accurate, due to the asymptotic convergence property. Subsequently, the number of moments can be decreased, increasing the computational efficiency of the algorithm.

(b) The number of moments need not be very high. As noted in [9], the first few moments of a distribution already contain significant information. In addition, we should keep in mind that the algorithm approximates the relaxed problem space, created by the relaxation algorithm. Discovering detailed information about the relaxed space does not necessarily increase the quality of information about the original problem domain. We must underline once more the importance of the selected relaxation algorithm. The noise introduced by a bad relaxation can not be effectively compensated by a multi-moment feature extraction algorithm.

(c) A careful study of the computation performed by a feature extraction algorithm indicates that a significant amount of computation time is consumed by the linear programming approximation module, which, given the moments, computes an approximation of the distribution function in the space of Bernstein polynomials [9]. Significant computational enhancements may be attained if a two-moment feature extraction algorithm is used and the PDFs are approximated by Gaussian distributions [9].

4. Scheduling test problem 2

The purpose of the second scheduling test is to investigate the global optimization characteristics of genetic algorithms and, consequently, the scheduling efficiency of the constant resolution strategy. The algorithm will be applied to an instance of the equipment and resource assignment subproblem, which involves the parallel production of six products in the model MBP. Table 18 contains the production requirements. The plant consists of 40 processing units, as shown in table 19. The resource availability levels are given in table 20. The objective of the scheduling effort is to minimize the makespan. This test problem is characterized by high decisional dimensionality and combinatorial complexity. It can be formulated as an MINLP involving 356 binary decision variables, 2402 continuous variables and 3657 linear and nonlinear constraints.

A preanalysis of the scheduling data provides some insights on the problem and the required solution strategy. In order to facilitate this task, we have relaxed the resource availability bounds. Since the production requirements of products B

Table 18

Production requirements for test problem 2.

Product	Quantity
A	55000
B	91000
C	32000
D	87000
E	27000
F	47000

Table 19

Processing units for test problem 2.

Equipment type	Number of units
Reactor U1	4
Reactor U2	4
Reactor V1	4
Reactor V2	4
Reactor D	5
Filter S	4
Filter P	7
Dryer T	4
Dryer P	4

Table 20

Resource availabilities for test problem 2.

Resource	Availability
S1	750
S2	500
S3	500
S4	800

and D are significantly higher than those of the remaining products, an efficient schedule would maximize the processing rates of these two products by distributing to the remaining products the least possible number of units. A closer look to the production time expressions indicates that the processing of B is the bottleneck of the whole production, because of the significant processing time required by its

fourth production task. Hence, an assignment of parallel in-phase equipment items to the production line of product B should be expected.

The problem was initially solved by the application of the focused successive refinement strategy. The results are presented in table 21 along with the required number of iterations and the corresponding computation times. To further illustrate the solution efficiency of the algorithm, the results are depicted in fig. 12, along with a randomly created reference solution. The objective function values of the obtained solutions are within 10 percent of the most efficient solution obtained in

Table 21

Results for test problem 2.

Cover population	Makespan	Iterations	CPU time (min)
3	2945	407	7
5	2695	610	17
10	2644	1355	38
15	2945	1978	56
30	2810	3745	106
reference	5383		

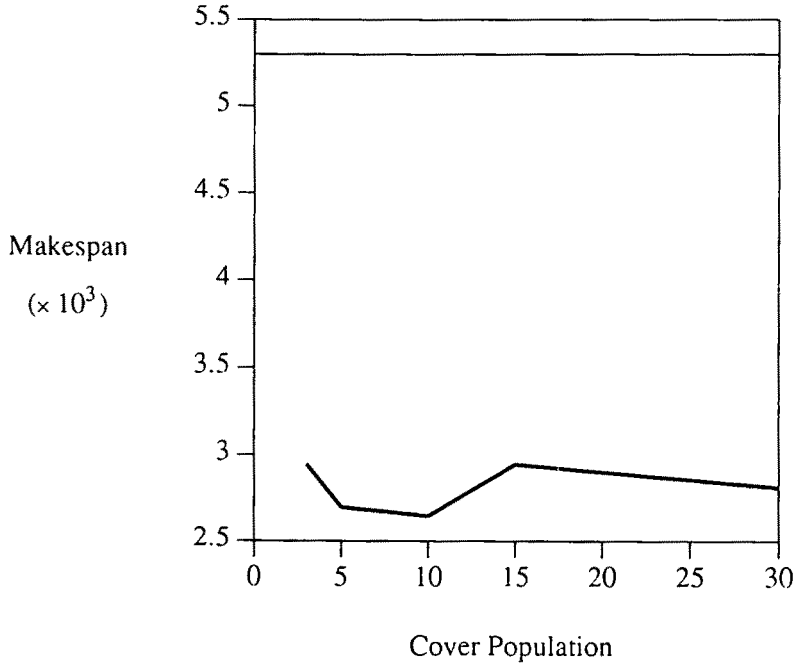

Fig. 12. Solution quality.

Table 22

Efficient solution for test problem 2 – part 1.

Product	Task	Equipment items
A	1	RD
	2	FS
	3	RU2
	4	RD
	5	RV1
	6	FP
	7	DT
B	1	RD
	2	FS
	3	RV1, RV2
	4	RU1, RU1
	5	RU1, RU2
	6	FP
	7	DT, DT

Table 23

Efficient solution for test problem 2 – part 2.

Order	Task	Equipment items
C	1	RD
	2	FS, FS
	3	RV2
	4	RD
	5	RV1
	6	FP
	7	DT
D	1	RU2
	2	RU2
	3	RV2
	4	FP
	5	DP, DP
E	1	RU1
	2	FP
	3	DP
F	1	RV1
	2	RV1
	3	FP, FP
	4	DP

all runs. Tables 22 and 23 contains the equipment assignment corresponding to the fifth solution of table 21. Evidently, the algorithm was able to identify product B as the operational bottleneck and assign to it multiple equipment items, in accordance with the preanalysis results. The algorithm is able to identify efficient solutions in very reasonable computation times. Evidently, the dynamics of genetic algorithms are well-suited for the MBP scheduling problems. It is possible that inefficient or infeasible strings contain efficient partial equipment assignments (e.g. individual production lines). Through cross-over the efficient production lines can be merged and a superior schedule created. The main disadvantage of genetic algorithms lies in their inability to guarantee the creation of feasible strings. These two themes (scheduling efficiency – influence of infeasibility) will be examined in detail.

In a general MINLP, the string has the following form:

$$\zeta = [X_1, X_2, \ldots, X_M, I_1, I_2, \ldots, I_K], \tag{22}$$

where the first M slots correspond to the binary decision variables, hence the X_i are evaluated to be either 0 or 1. The remaining K slots correspond to the continuous problem variables. I_j contains intervals from the range of the corresponding continuous variable. In the present implementation, the solution effort will be focused on the binary assignment variables. Therefore, the continuous variables will remain uninstantiated. The string fitness is evaluated using eq. (26) in [9]:

$$\mu\delta\Omega) = \bar{f}(\delta\Omega) + R \sum_{i=1}^{K+L} (1 - \rho_i(\delta\Omega)). \tag{23}$$

Infeasible strings are acceptable. In order to be meaningful, the solutions contained in the strings must satisfy logical constraint (3): During the genetic search it is possible that equipment assignments will be created, in which no equipment units are assigned to a particular production task. This scenario creates a singularity in the objective average calculation: if no units are assigned to a task, then the production can not be satisfied in any finite interval. This inefficiency is accommodated by conveniently redefining the genetic operators. If the string that was created by mutation or cross-over is singular, then a non-zero assignment is randomly selected for the singular production task. For the particular scheduling problem the population cardinality is set equal to 50, the penalty function multiplier R is 1000 and the genetic operators are selected with equal probabilities.

The results of the application of the constant resolution strategy with the second test problem are presented in table 24. The graph in fig. 13 depicts the dependence of the solution quality on the invested computational effort. The obtained results reveal the inherent inefficiency of genetic algorithms in searching highly constrained domains. Analyzing fig. 13, one can easily identify two phases in the genetic search. During the first phase, the search effort is focused on increasing the feasibility of the string populations. Small changes in the string feasibility result in significant change in the global optimality measure (23). As a consequence, little

Table 24

Constant resolution strategy – original mutation.

Number of generations	Makespan	CPU time (min)
100	INFES	106
200	INFES	212
300	INFES	318
400	INFES	424
500	INFES	530
600	INFES	636
800	INFES	848
1000	2945	1060
1600	2945	1700

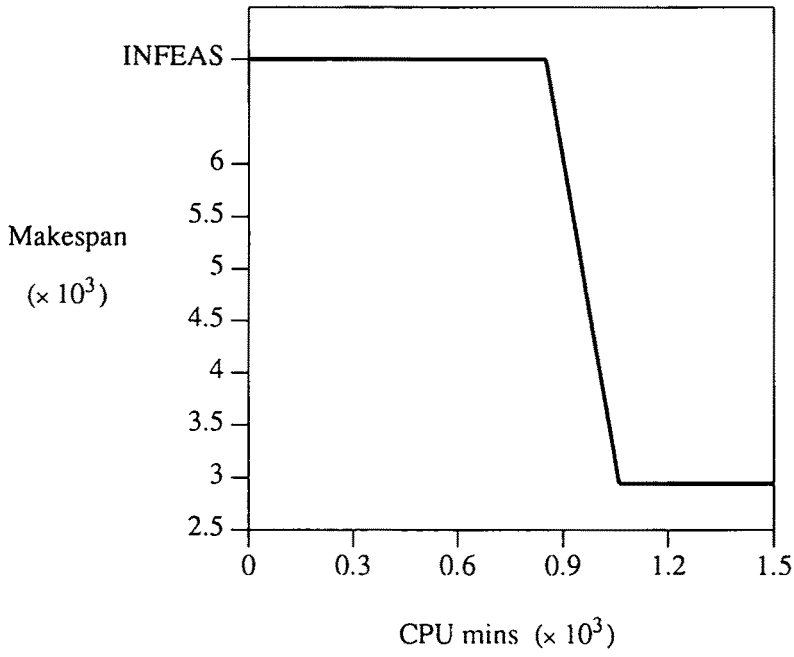

Fig. 13. Computation times of constant resolution strategy.

or no attention is paid to the relative objective average differences among the population members. The second phase is initiated when the magnitude of the infeasibility measure in (23) has been significantly reduced. At this point, the string population contains nearly feasible solutions and the string fitness is almost exclusively determined by the objective average performance of the corresponding solution.

During the second search phase, locally optimal assignments are combined to create globally efficient solutions. The newly created populations are inhibited from moving in the infeasible region by the fitness-proportionate selection rule.

The identification of a feasible equipment assignment requires more than 800 generations, or, equivalently, 850 cpu minutes, whereas the remaining 200 cpu minutes are devoted to the search for an efficient solution. The constant resolution strategy requires at least one order of magnitude more computation time than the successive refinement approach, for the same quality of solutions. The reason for this difference lies in the ability of the successive refinement strategy to create coverings of the feasible space. Every iteration of this algorithm results in identifying and removing infeasible parts of the feasible region. Naturally, the search is directed towards a space of increasing feasibility.

Since at least eighty percent of the computation is spent on increasing the feasibility of the string population, we decided to reformulate the mutation operator in an effort to increase the feasibility of the string population, without sacrificing the global optimization characteristics of the genetic search. In essence, a variable X_{pme} that is selected for mutation will be able to choose a value from the set {0, 1}, 1 only if equipment item e has not yet been assigned to any other production task. Otherwise, it will be forced to 0. The modification search contains two stages:

(a) In the first stage, heuristic mutation is chosen with a probability of 1 and the string selection is random. This phase is equivalent to a preprocessing step that increases the feasibility of the string population. It continues for a prespecified number of generations. The selection is not fitness-proportionate so that the newly created populations remain as diverse as possible.

(b) In the second stage, the genetic operators are applied to the modified string population with equal probabilities. The user is free to select between the heuristic and the original mutation operator.

Table 25 presents the results obtained by the modified genetic search on the second test problem. The preprocessing phase lasted for 200 generations. The heuristic

Table 25

Constant resolution strategy – original mutation.

Number of generations	Makespan	CPU time (min)
100	INFES	71
200	5312	141
300	2719	247
400	2719	424
500	2719	496
600	2719	600

mutation operator was utilized during the second stage. The results are also shown in fig. 14. The results obtained by the original genetic search are superimposed for ease of comparison. The positive influence of the heuristic mutation operator on the computational efficiency of the genetic algorithm search is clearly demonstrated.

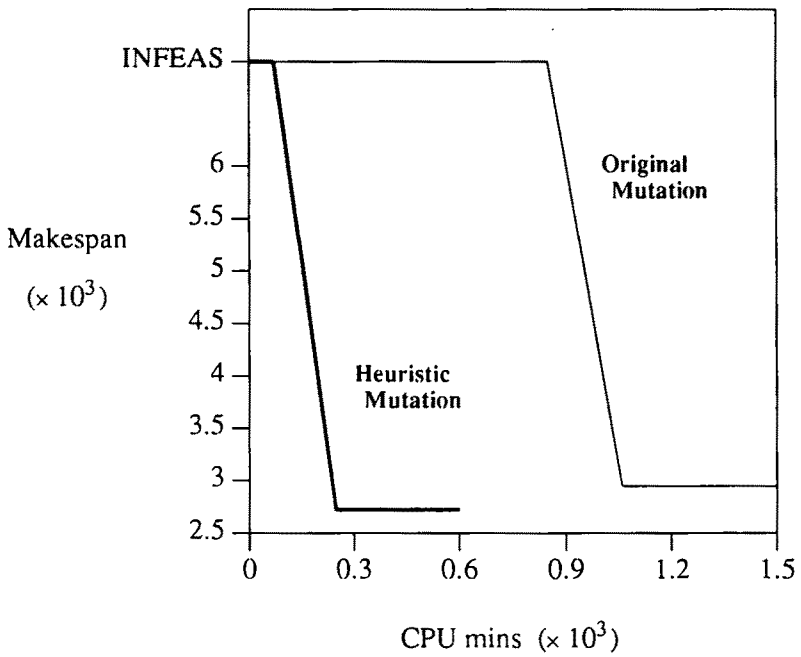

Fig. 14. Comparison of computation efficiency.

The computational effort that is necessary to increase the feasibility of the string population has been reduced by a factor of four. In the next 100 generations, the combination of mutation and cross-over were able to significantly increase the efficiency of the discovered solutions.

The conventional application of genetic algorithms to a MINLP problem requires a solution of the reduced nonlinear optimization problem for each newly created string [1]. The objective value of the discovered local optimum is the fitness of the corresponding string. For the second test problem, the solution of the reduced nonlinear programming model using MINOS version 5.3, executed under the mathematical representation language GAMS version 2.25, requires on the average 30 cpu seconds on a SUN SPARCstation 1. By contrast, the computation of string fitness by the feature extraction approach requires 0.85 cpu seconds on the same machine. Hence, the constant resolution strategy is approximately 35 times faster than the conventional genetic algorithmic search, without any significant sacrifice in the obtained solution quality, as is evident from figure 14. It is clear that the

feature extraction algorithms significantly extend our ability to solve problems of practical combinatorial complexity in reasonable computation times.

4.3. DISCUSSION

The analysis of the second test example indicates that the feature extraction approach significantly extends the limits of applicability of genetic algorithms. The results obtained suggest that the ability of genetic algorithms to identify globally efficient solutions should not be underestimated. Unfortunately, the genetic operators are not well suited for constrained problem domains, such as the MBP scheduling problem. As a result, their application required significant preanalysis and heuristic modifications, which can be performed only by a user with considerable experience with the problem at hand. Once again, the focused successive refinement approach seems to offer a very attractive solution alternative.

References

[1] I.P. Androulakis and V. Venkatasubramanian, A genetic algorithmic framework for process design and optimization, Comp. Chem. Eng. 15(1991)217–228.

[2] A. Brook, D. Kendrick and A. Meeraus, *GAMS, A User's Guide* (Scientific Press, Redwood City, CA, 1988).

[3] B.A. Murtagh and M.A. Saunders, *MINOS 5.0 USER's GUIDE*, Technical Report SOL 83-20, Stanford University Systems Optimization Laboratory (December 1983).

[4] K.G. Murty and S.N. Kabadi, Some NP-complete problems in quadratic and nonlinear programming, Math. Progr. 39(1987)117.

[5] A. Papoulis, *Probability, Random Variables and Stochastic Processes*, 2nd ed. (McGraw–Hill, New York, 1984).

[6] H. Ratschek, Inclusion functions and global optimization, Math. Progr. 33(1985)300–317.

[7] G.V. Reklaitis, Perspectives on scheduling and planning of process operations, *4th Int. Symp. on Process Systems Engineering*, Montebello, Canada (August, 1991).

[8] A.G. Tsirukis, M. Zentner, J.F. Pekny and G.V. Reklaitis, Scheduling and planning of process operations, (1993) in preparation.

[9] A.G. Tsirukis and G.V. Reklaitis, Feature extraction algorithms for global optimization: I. Mathematical foundations, Ann Oper. Res. (1993), this volume.

[10] M.C. Wellons and G.V. Reklaitis, Scheduling of multipurpose batch plants – Part 2. Multiple-product campaign formation and production planning, Ind. Eng. Chem. Res. 30(1991)688–705.